

二项堆

5.1 二项堆

5.1.1 二项堆的引入与基本概念

二项堆的引入来源于我们希望连续插入的操作有常数的摊还时间。回忆二叉堆可以一次读入全部元素，在 $O(n)$ 时间内建堆，但依次调用普通 Insert 并没有这样的保证。左式堆和斜堆也可以线性建堆，例如先在线性时间内建立完全二叉最小堆，再改用指针存储（注意完全二叉树满足左式性质，补算 NPL 也只需线性时间）。本节则希望在元素逐个到来时，仍能保证从空堆开始的 n 次 Insert 总共只需要 $O(n)$ 时间，同时保持合并的对数时间。

二项堆的定义如下：

1. 结构性质：

- (a) 二项堆由多棵树构成一个森林，其中每一棵树都称为二项树；
- (b) 一个二项堆中的每棵二项树具有不同的高度，即每一个高度最多对应一棵二项树；
- (c) 高度为 0 的二项树是一棵单结点树；高度为 k 的二项树 B_k 通过将一棵二项树 B_{k-1} 附接到另一棵二项树 B_{k-1} 的根上而构成，图 5.1 给出了前几棵二项树。

2. 序性质：每棵二项树都保持堆的序性质，例如是最小堆（本节无特殊说明都默认最小堆）则根结点最小，孩子的键都不小于父亲的键。

进一步观察会发现更有趣的事情：二项树 B_k 实际上就是由一个带有儿子 $B_0, B_1, \dots, B_{k-1}$ 的根组成。根据定义，可以归纳得到下面的结论。

定理 5.1 (二项树的结点数). 对任意整数 $k \geq 0$ ，二项树 B_k 恰好有 2^k 个结点，其中深度为 d 的结点恰好有 $\binom{k}{d}$ 个 ($0 \leq d \leq k$)。

证明. 数学归纳法。 B_0 只有一个根结点，结论成立。假设结论对 B_k 成立。由于 B_{k+1} 由两棵 B_k 连接而成，其结点总数为 $2 \cdot 2^k = 2^{k+1}$ 。连接后，一棵树中各结点的深度保持不变，另一棵树中各结点的深度增加 1，因此对 $1 \leq d \leq k$ ，深度为 d 的结点数为

$$\binom{k}{d} + \binom{k}{d-1} = \binom{k+1}{d}.$$

深度为 0 和 $k+1$ 的结点都只有一个，也分别等于 $\binom{k+1}{0}$ 和 $\binom{k+1}{k+1}$ ，命题得证。 $\square$

由此可见，各深度的结点数恰好是二项系数，因此二项树的名字得来非常自然。

根据二项堆结构性性质，每种高度至多出现一次。因此具有 n 个结点的二项堆恰好对应 n 的二进制表示：第 k 位为 1 当且仅当堆中含有 B_k 。例如 $13 = (1101)_2 = 8 + 4 + 1$ ，堆中就有 B_3, B_2, B_0 ，没有 B_1 。并且不难理解，非空堆至多有 $\lfloor \log_2 n \rfloor + 1$ 棵树。

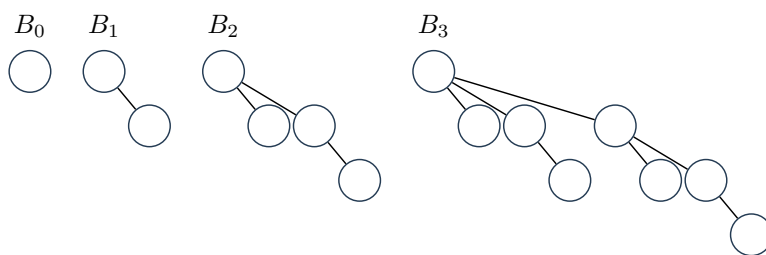


图 5.1 二项树 B_0 至 B_3 。每次把两棵同阶树的根相连，结点数加倍，高度增加 1。

5.1.2 二项堆的操作与分析

最简单的操作是 FindMin，直接遍历全部 $O(\log n)$ 棵树（回忆二进制表示）的根结点即可，因此时间复杂度是 $O(\log n)$ 。当然也可以通过专门记录最小的根结点来实现 $O(1)$ 的时间复杂度，但必须在每次可能改变最小元素的操作后维护这一指针。

然后是插入操作，和左式堆一样，插入可以看作特殊的合并，因此先分析合并操作。实际上两个二项堆的合并想法也非常简单：既然每个二项堆都和唯一的二进制数对应，那合并后二项堆中二项树的分布情况不就符合二进制数的加法后的结果吗？因此合并操作实际上就是从最低阶的二项树开始（对应二进制最低位），如果无需进位（表明是 $1 + 0$ 或 $0 + 1$ ）则直接留下作为新堆的一部分，如果需要进位则表明是 $1 + 1$ ，因此做合并后进位与下一位做加法，如此循环直到最高位完成操作。时间复杂度分析非常简单，在保证堆的存储顺序是按高度从小到大排列的前提下，时间复杂度很显然就是 $O(\log n)$ ，因为就是二进制逐位做操作。

插入就是与一棵 B_0 合并，因此最坏时间为 $O(\log n)$ 。需要特别注意的是，从空堆连续插入 n 个元素的总时间为 $O(n)$ ，因而每次 Insert 的摊还时间为 $O(1)$ 。这与例 1.5 中二进制计数器连续加一的问题完全对应：第 i 位为 1，表示堆中有一棵 B_i ；插入时，若已有同阶树，就将两棵树链接成高一阶的树，对应当前位从 1 变为 0 并向高位进位；遇到空位后放入进位树，对应这一位从 0 变为 1，本次插入结束（注意插入只是将一个结点加入，因此只要遇到某一位加法无需进位，就可以马上停止插入）。注意每次链接或放入树都只需常数时间，而例 1.5 已证明从 0 开始连续加一 n 次，总翻转位数不超过 $2n$ ，所以加上每次新建结点的常数开销，连续插入的总时间仍为 $O(n)$ 。

DeleteMin 先扫描各棵树的根找到最小值。设它是 B_k 的根，删除它后得到两个二项堆：一个是原堆移去整棵 B_k 后剩下的森林，另一个是被删根的孩子 $B_0, \dots, B_{k-1}$ ，将这两个堆合并即可，总时间仍是 $O(\log n)$ 。

图 5.2 中展示了合并 $6 = (110)_2$ 与 $7 = (111)_2$ 的结果。第 0 位留下 13；第 1 位把根为 16、14 的两棵树链接，产生一个 B_2 。第 2 位有三棵树，留下刚才的进位树，再把其它两棵输入树链接成 B_3 ，得到的结果与 $13 = (1101)_2$ 一致。最后也给出了进一步删除最小根 12 后的结果。

后面讨论堆加速 Dijkstra 算法时，需要 DecreaseKey 操作，事实上这在二项堆中是再简单不过的事情，因为每棵树都是二项树，所以类似于普通二叉堆的 DecreaseKey 操作，只需要在违反序性质时将做了 DecreaseKey 的记录与父结点中的记录不断交换直到恢复序性质（percolate up）即可，时间复杂度显然也是 $O(\log n)$ 的。删除任意结点操作显然也是和普通二叉堆一样顺其自然可以支持

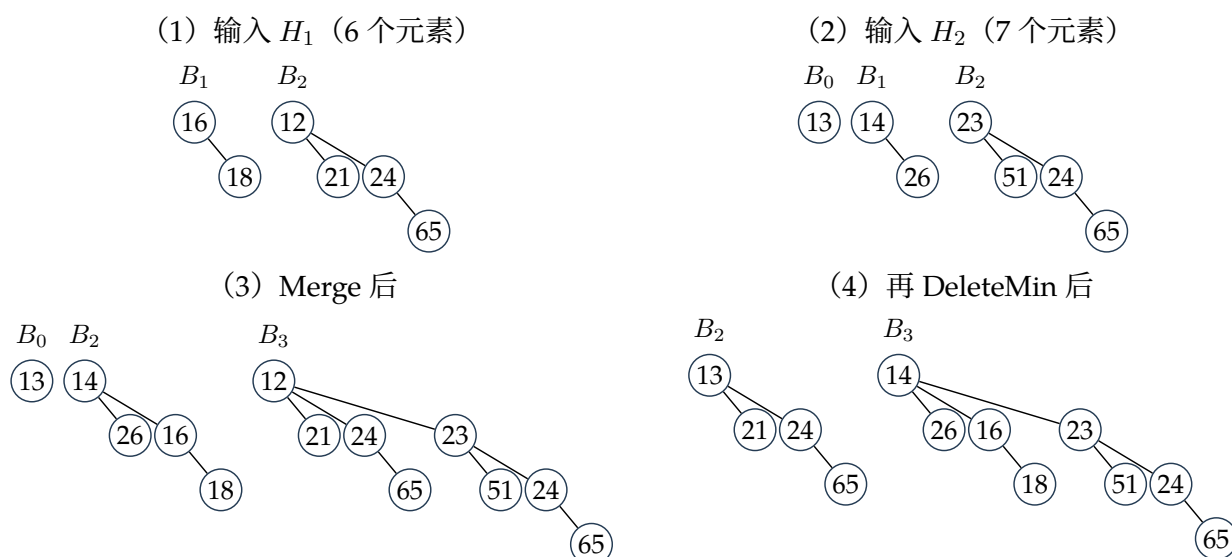
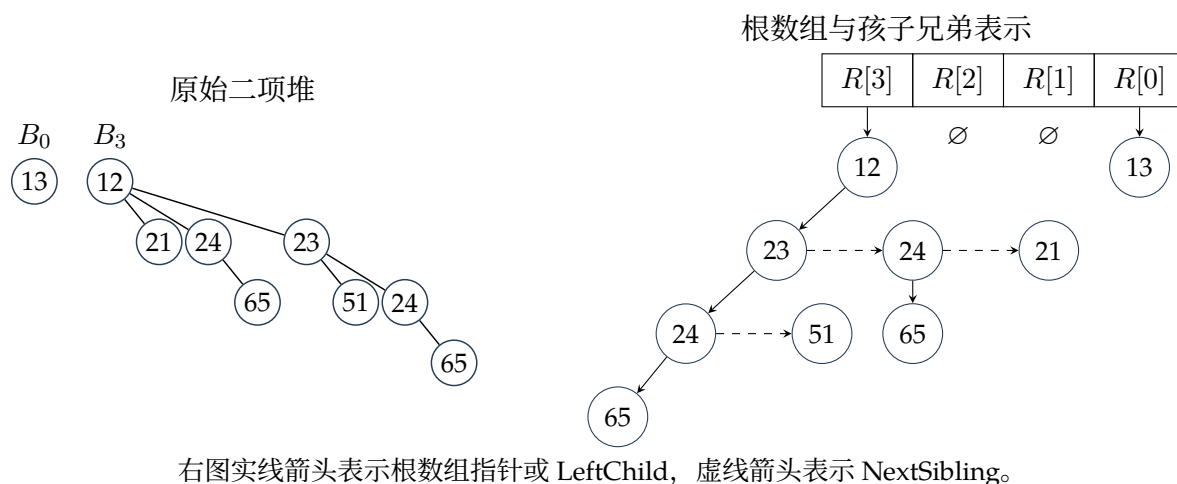


图 5.2 二项堆的合并与删除最小值，重复的 24 和 65 是不同结点。

的，就是给目标记录加上一个优先于所有普通键的删除标记，按 DecreaseKey 的方式上滤到根，再调用 DeleteMin 即可，显然时间复杂度为 $O(\log n)$ 。

5.1.3 二项堆的代码实现

虽然二项堆的操作描述起来非常容易，但具体实现时有一些细节问题需要解决。最关键的问题在于每个结点的孩子数量可能不只有 2 个，因此我们使用 LeftChild 和 NextSibling 的组合实现。直观上来看用 LeftChild 和 NextSibling 是让二项树翻转了（如图 5.3 所示）：原先是根的子树从左到右高度依次增大，现在依次减小了。并且为了方便索引每棵二项树，用一个数组存储每棵二项树的根，其中数组的索引就对应二项树的高度。



右图实线箭头表示根数组指针或 LeftChild，虚线箭头表示 NextSibling。

图 5.3 同一个二项堆的树形与孩子兄弟表示。12 的孩子在右图中按降阶顺序排列为 23、24、21。

因为插入、DeleteMin 的关键操作都是合并，因此下面主要讨论合并，而合并中经常需要合并两棵大小相同的二项树，因此首先介绍这一操作（称为 CombineTrees）的实现。在 CombineTrees 合并两棵相同大小的二项树时，结合堆的序性质以及 LeftChild 和 NextSibling 的实现，很容易想到其实只需要直接用根结点小的根作为新的根，根结点大的整棵树作为 LeftChild，其 NextSibling 接到原先的 LeftChild 即可（可以结合图 5.3 理解）。

算法 5.1 合并两棵同阶树 CombineTrees(T_1, T_2)**输入：** 两棵非空同阶二项树的根**输出：** 返回合并后的根；键相等时保留 T_1 为根

```

1: if  $T_1.Element > T_2.Element$  then
2:   return CombineTrees( $T_2, T_1$ )
3: end if
4:  $T_2.NextSibling \leftarrow T_1.LeftChild$ 
5:  $T_1.LeftChild \leftarrow T_2$ 
6: return  $T_1$ 

```

有了 CombineTrees，就可以按二进制加法逐阶合并两个堆。

算法 5.2 二项堆 Merge(H_1, H_2)**输入：** 两个二项堆 H_1, H_2 ，根数组容量相同**输出：** 返回合并后的 H_1 ，并清空 H_2 ；容量不足时报告错误，两个堆保持不变

```

1: if  $H_1.CurrentSize + H_2.CurrentSize > Capacity$  then
2:   报告容量不足并结束
3: end if
4:  $H_1.CurrentSize \leftarrow H_1.CurrentSize + H_2.CurrentSize$ 
5:  $Carry \leftarrow null; i \leftarrow 0; j \leftarrow 1$ 
6: while  $j \leq H_1.CurrentSize$  do
7:    $T_1 \leftarrow H_1.TheTrees[i]; T_2 \leftarrow H_2.TheTrees[i]$ 
8:   switch  $4*!!Carry + 2*!!T_2 + !!T_1$ 
9:     case 0, 1: break
10:    case 2:  $H_1.TheTrees[i] \leftarrow T_2$ 
11:       $H_2.TheTrees[i] \leftarrow null; \mathbf{break}$ 
12:    case 4:  $H_1.TheTrees[i] \leftarrow Carry$ 
13:       $Carry \leftarrow null; \mathbf{break}$ 
14:    case 3:  $Carry \leftarrow \text{CombineTrees}(T_1, T_2)$ 
15:       $H_1.TheTrees[i] \leftarrow null; H_2.TheTrees[i] \leftarrow null; \mathbf{break}$ 
16:    case 5:  $Carry \leftarrow \text{CombineTrees}(T_1, Carry)$ 
17:       $H_1.TheTrees[i] \leftarrow null; \mathbf{break}$ 
18:    case 6:  $Carry \leftarrow \text{CombineTrees}(T_2, Carry)$ 
19:       $H_2.TheTrees[i] \leftarrow null; \mathbf{break}$ 
20:    case 7:  $H_1.TheTrees[i] \leftarrow Carry$ 
21:       $Carry \leftarrow \text{CombineTrees}(T_1, T_2)$ 
22:       $H_2.TheTrees[i] \leftarrow null; \mathbf{break}$ 
23:   end switch
24:    $i \leftarrow i + 1; j \leftarrow 2j$ 
25: end while

```

```

26:  $H_2.CurrentSize \leftarrow 0$ 
27: return  $H_1$ 

```

第 i 轮需要处理的树最多有三棵： H_1 中的 T_1 、 H_2 中的 T_2 ，以及进位树 $Carry$ ，它们都是 B_i 。每棵树各有存在和不存在两种情况，因此共有 $2^3 = 8$ 种组合。代码用 C 语言中的双重取非 `!!` 将指针转为 0 或 1：空指针对应 0，非空指针对应 1。所以 $4*!!Carry + 2*!!T_2 + !!T_1$ 就把这三棵树的存在情况编码为一个三位二进制数，从高位到低位依次对应 $Carry, T_2, T_1$ 。整体思路非常简单：当前阶有零棵或一棵树时，无需合并；有两棵时，将它们合为一棵 B_{i+1} ，向下一阶进位；有三棵时，在当前阶留下一棵，其余两棵合并后进位，具体情况如下：

1. 0 (000)：三棵树都不存在，当前阶保持为空。
2. 1 (001)：只有 T_1 ，它已经在 H_1 中，无需修改。
3. 2 (010)：只有 T_2 ，将它移入 $H_1.TheTrees[i]$ ，并清空 H_2 的对应位置。
4. 3 (011)：将 T_1, T_2 合并为进位树，并清空两个堆的当前阶。
5. 4 (100)：只有 $Carry$ ，将它放入 $H_1.TheTrees[i]$ ，然后清空 $Carry$ 。
6. 5 (101)：将 T_1 与 $Carry$ 合并为新的进位树，并清空 H_1 的当前阶。
7. 6 (110)：将 T_2 与 $Carry$ 合并为新的进位树，并清空 H_2 的当前阶。
8. 7 (111)：将原来的 $Carry$ 留在 H_1 的当前阶，再将 T_1, T_2 合并为新的进位树，并清空 H_2 的当前阶。

5.2 堆加速 Dijkstra 算法

5.2.1 回顾 Dijkstra 算法

Dijkstra 算法用于求解边权非负的单源最短路径问题。设图为 $G = (V, E)$ ，源点为 s ，我们希望求出 s 到每个顶点的最短距离。下面按有向图描述算法；对于无向图，只需把一条无向边看成两条方向相反、权重相同的边。

算法把顶点分成两类：集合 X 中的顶点已经确定了最短距离，其余顶点还在等待处理。对于每个顶点 v ，用 $d[v]$ 记录目前找到的从 s 到 v 的最短路径长度，称为暂定距离。这里“暂定”的意思是，后面可能还会找到更短的路径；如果目前没有找到任何路径，就令 $d[v] = +\infty$ 。初始时，令 $d[s] = 0$ ，其余顶点的暂定距离为 $+\infty$ ，并令 X 为空。每一轮从 $V \setminus X$ 中选出暂定距离最小的顶点 u ，将 $d[u]$ 确定为它的最短距离，再把 u 加入 X 。随后检查 u 到各个未确定顶点的出边：如果 $d[u] + w(u, v) < d[v]$ ，就说明经过 u 可以得到一条更短的路径，于是将 $d[v]$ 更新为 $d[u] + w(u, v)$ ，并用 $pred[v] = u$ 记下这条路径中 v 的前驱。这个尝试改进距离的过程称为松弛。如果剩余顶点中最小的暂定距离已经是 $+\infty$ ，则它们都无法从源点到达，算法结束。

以图 5.4 为例，从 s 出发，首先得到 $d[a] = 10$ 、 $d[b] = 3$ ，因此下一轮选择 b 。通过 b 的出边，可以把 a, c, t 的暂定距离分别更新为 4, 5, 12，所以下一个确定的是 a 。处理 a 时，经过它到达 c 的路径长度为 6，不能改进已有的 5，但到达 t 的路径长度为 11，可以将原来的 12 改小。接着确定 c ，又把 $d[t]$ 降为 7。最终确定到 t 的最短路径为 $s \rightarrow b \rightarrow c \rightarrow t$ 。

正确性. 下面证明，每轮选出的顶点 u 都满足 $d[u] = \delta(s, u)$ ，其中 $\delta(s, u)$ 表示真正最短距离。第

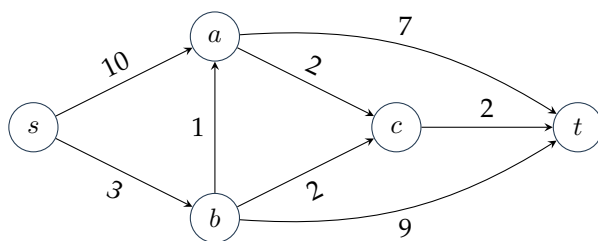


图 5.4 Dijkstra 算法的示例，边旁的数字表示权重。

一轮选出源点 s ，结论成立。假设已经加入 X 的顶点都满足结论，考虑接下来选出的可达顶点 u 。

取一条从 s 到 u 的最短路径，并设 y 是这条路径上第一个不在 X 中的顶点， x 是它在路径上的前驱。由于 $x \in X$ ，处理 x 时已经用正确距离 $d[x] = \delta(s, x)$ 松弛过边 (x, y) ，因而得到了 $d[y] = \delta(s, y)$ 。这里用到了最短路径的前缀仍是最短路径：如果存在更短的前缀，用它替换原来的前缀，就会得到一条更短的 s 到 u 的路径，与选择矛盾。

因为边权非负，从 y 继续走到 u 不会使路径长度减小，所以 $\delta(s, y) \leq \delta(s, u)$ 。另一方面，算法选出的 u 在所有未确定顶点中具有最小的暂定距离，因此 $d[u] \leq d[y]$ 。由于暂定距离不小于真正最短距离，得到 $\delta(s, u) \leq d[u] \leq d[y] = \delta(s, y) \leq \delta(s, u)$ 。两端相等，说明中间各项也都相等，特别是 $d[u] = \delta(s, u)$ ，归纳完成。

如果尚有可达顶点未被确定，沿它的一条最短路径，同样可以找到第一个不在 X 中的顶点 y ；它的前驱已经被处理，所以 $d[y]$ 必为有限值。因此，当所有剩余暂定距离都是 $+\infty$ 时，剩下的顶点确实都不可达。□

5.2.2 用堆实现未确定顶点集合

现在回到本讲关心的问题：堆究竟能加速 Dijkstra 的哪一步？每一轮都要从 $V \setminus X$ 中找到暂定距离最小的顶点，而松弛又会降低其中一些顶点的暂定距离。因此，可以把所有未确定顶点存入最小堆，以 $d[v]$ 为键，用 DeleteMin 选出下一轮的顶点，用 DecreaseKey 完成距离更新。

算法 5.3 堆实现的 Dijkstra(G, s)

输入：邻接表图，所有边权非负， $s \in V$

输出：距离 d 与前驱 $pred$ ；不可达顶点距离为 $+\infty$

- 1: 对每个 v ，置 $d[v] \leftarrow +\infty$ 、 $pred[v] \leftarrow null$ 、 $done[v] \leftarrow false$
- 2: $d[s] \leftarrow 0$ ；建堆 H ，每个顶点一条记录 $(v, d[v])$
- 3: **while** H 非空 **do**
- 4: $u \leftarrow \text{DeleteMin}(H).id$
- 5: **if** $d[u] = +\infty$ **then**
- 6: **break**
- 7: **end if**
- 8: $done[u] \leftarrow true$
- 9: **for** 每条出边 (u, v) **do**
- 10: **if** $\neg done[v]$ 且 $d[u] + w(u, v) < d[v]$ **then**
- 11: $d[v] \leftarrow d[u] + w(u, v)$ ； $pred[v] \leftarrow u$

```

12:     DecreaseKey( $H, v, d[v]$ )
13:   end if
14: end for
15: end while
16: return ( $d, pred$ )

```

伪代码中的 $done[v]$ 表示 v 是否已经加入 X 。设 $n = |V|$ 、 $m = |E|$ 。每个顶点最多取出一次，每条边最多检查一次，每次检查至多引起一次降键。因此，若建堆、一次 DecreaseKey 和一次 DeleteMin 的时间分别记为 $T_{\text{build}}(n)$ 、 T_{dk} 和 T_{em} ，总时间就是

$$O(n + m) + T_{\text{build}}(n) + O(mT_{\text{dk}} + nT_{\text{em}}).$$

这个式子也说明了不同数据结构对算法的影响：

1. 如果用数组保存暂定距离，更新一个距离只需 $O(1)$ 时间，但每轮都要扫描尚未确定的顶点寻找最小值，最多花 $O(n)$ 时间。因此总时间为 $O(m + n^2) = O(n^2)$ 。
2. 如果使用堆，DeleteMin 和 DecreaseKey 都只需 $O(\log n)$ 时间，故总时间为 $O((m + n) \log n)$ 。当 $n \geq 2$ 且所有顶点都能从源点到达时，必有 $m \geq n - 1$ ，此时也可以写成 $O(m \log n)$ 。

对于边数较少的图（稀疏图），具体地，当 $m = o(n^2 / \log n)$ 时，上面的堆实现时间界为 $o(n^2)$ ，优于数组版本的二次上界。在现实的路网、通信网等图中，边数通常远小于顶点数的平方，因此堆实现的 Dijkstra 算法在这些场景下更有优势。此外，有些堆实现只提供 Insert 和 DeleteMin，没有 DecreaseKey。这时也可以在每次距离变小时插入一条新记录，并暂时保留旧记录。例如， $d[v]$ 从 12 降为 7 时，在堆中加入 $(7, v)$ ，原来的 $(12, v)$ 留到以后取出时再丢弃。

5.3 斐波那契堆

在 Dijkstra 算法中，DecreaseKey 最多执行 m 次，DeleteMin 最多执行 n 次。如果能把 DecreaseKey 的摊还时间降为 $O(1)$ ，同时让 DeleteMin 保持对数摊还时间，总时间就能降为 $O(m + n \log n)$ 。斐波那契堆正是为了获得这样的时间复杂度而设计的。

5.3.1 基本结构

斐波那契堆也是由若干棵满足最小堆序的树组成的森林。与二项堆相比，它放宽了对树形的要求：各棵树不必是二项树，也允许多个根具有相同的度数（度数指结点的孩子数）。每个父结点的键仍然不大于孩子的键，因此全堆最小键一定出现在某个根上。所有根用一条循环双向链表连接，称为根链表；同一个结点的所有孩子也用一条循环双向链表连接，称为它的孩子链表，链表中的结点不需要按键值或度数排序。堆本身保存根链表 $H.roots$ 、最小根指针 $H.min$ 和结点总数 $H.n$ 。每个结点需要保存以下信息：

- 记录，其中用来比较大小的值称为键，记为 key 。例如 Dijkstra 算法中，记录 $(7, v)$ 表示顶点 v 当前的暂定距离为 7，其中 7 是键， v 说明这个距离属于哪个顶点。取出最小值时需要返回整条记录，才能知道接下来处理哪个顶点。如果只需保存数值，记录就是数值本身。
- 指向父结点的指针 $parent$ 。

- *child* 指向任意一个孩子，没有孩子时为 *null*，*degree* 记录孩子数。
- *left, right* 指向同一条链表中的前驱和后继。链表只有一个结点时，这两个指针都指向结点自身。
- 布尔标记 *mark*，用来记录一个非根结点自从最近一次成为孩子以来，是否已经失去过一个孩子。这是斐波那契堆的一个重要特性，它约束了结点失去孩子的次数。一个结点刚被接到另一个结点下面时，标记为 *false*；它第一次失去孩子时，将标记置为 *true*；第二次失去孩子时，就把它连同剩余子树一起从父亲处切下，放入根链表。这样，**任何一个仍是非根的结点，自从最近一次成为孩子以来，至多失去过一个孩子**。本讲约定根的标记始终为 *false*，后面将证明，这条规则足以保证度数为 *k* 的结点拥有随 *k* 指数增长的子树。

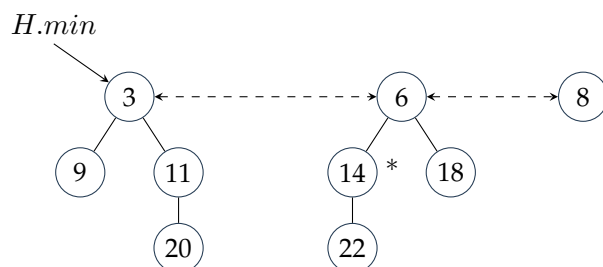


图 5.5 一个斐波那契堆；星号表示带标记的结点。

图 5.5 中共有 9 个结点，结点 14 上的星号表示它带有标记，其余结点都没有标记。树边表示父子关系，虚线示意根链表；为使图形清楚，省略了链表的回环以及孩子之间的链表指针。

5.3.2 插入、查找最小值与合并

插入时，新建一个没有孩子的根，将它加入根链表，再更新最小根即可。二项堆在此时需要处理同阶树之间的进位，斐波那契堆允许根的度数重复，因而暂时不做合并。

算法 5.4 $\text{FibonacciInsert}(H, r)$

输入：堆 H ；待插入记录 r ，其键为 $r.key$

输出：插入记录并返回其结点指针

- 1: 新建结点 x ，保存记录 r ， $x.key \leftarrow r.key$
- 2: $x.parent \leftarrow null$ ； $x.child \leftarrow null$ ； $x.degree \leftarrow 0$ ； $x.mark \leftarrow false$ ；将 x 加入 $H.roots$
- 3: **if** $H.min = null$ 或 $x.key < H.min.key$ **then**
- 4: $H.min \leftarrow x$
- 5: **end if**
- 6: $H.n \leftarrow H.n + 1$ ；**return** x

FindMin 直接访问 $H.min$ 即可，略去伪代码。合并两个堆时，只需拼接根链表，并在两个原最小根中选出新的最小根。

算法 5.5 $\text{FibonacciMerge}(H_1, H_2)$

输入：两个不同的堆

输出：返回合并后的堆 H ，并将 H_1, H_2 置空

- 1: 建立空堆 H ，将 $H_1.roots, H_2.roots$ 拼接为 $H.roots$

```

2:  $H.n \leftarrow H_1.n + H_2.n$ ;  $H.min \leftarrow H_1.min$ 
3: if  $H.min = null$  then
4:    $H.min \leftarrow H_2.min$ 
5: else if  $H_2.min \neq null$  且  $H_2.min.key < H.min.key$  then
6:    $H.min \leftarrow H_2.min$ 
7: end if
8:  $H_1.roots, H_1.min, H_2.roots, H_2.min \leftarrow null$ ;  $H_1.n, H_2.n \leftarrow 0$ 
9: return  $H$ 

```

这三个操作都只进行常数次比较和链表操作，实际时间均为 $O(1)$ 。代价是根链表可能越来越长，例如连续插入 n 个元素后，就有 n 个互不连接的单结点树。

5.3.3 删除最小值：合并度数相同的根

DeleteMin 删除一个键最小的结点，并返回其中保存的记录。删除最小根 z 后，它原来的每个孩子都成为一棵树的根。将这些孩子加入根链表，重新寻找最小值；我们就在这次遍历中合并度数相同的根，让后续操作面对较少的树。

算法 5.6 FibonacciDeleteMin(H)

输入： 斐波那契堆 H

输出： 删除并返回一条键最小的记录；空堆返回“无元素”

```

1: if  $H.min = null$  then
2:   return “无元素”
3: end if
4:  $z \leftarrow H.min$ ; 保存  $z$  中的记录  $r$ ; 从  $H.roots$  摘下  $z$ ;  $H.min \leftarrow null$ 
5: while  $z.child \neq null$  do
6:   从  $z$  的孩子链表摘下一个结点  $x$ ;  $z.degree \leftarrow z.degree - 1$ 
7:    $x.parent \leftarrow null$ ;  $x.mark \leftarrow false$ ; 将  $x$  加入  $H.roots$ 
8: end while
9: 释放结点  $z$ ;  $H.n \leftarrow H.n - 1$ 
10: Consolidate( $H$ ); return  $r$ 

```

▷ z 的指针失效

整理过程 Consolidate 需要使用一个数组 A ，其中 $A[d]$ 保存一个已经处理过、度数为 d 的根。如果处理当前根 x 时发现 $A[d]$ 已被占用，就把两个根中键较大的一个接到较小的一个下面。新根的度数增加了 1，继续检查下一个位置，直到找到空位。这与二项堆合并时的进位相同，不过这里必须先遍历无序的根链表，才能发现哪些度数发生了重复。一个细节问题是数组 A 的大小无法确定，事实上可以使用动态数组实现：访问下标超出容量时将容量不断加倍。

算法 5.7 Consolidate(H)

输入： H 的根链表保存待整理的全部树， $H.min$ 可以尚未确定

输出： 每个度数至多有一个根，并重新确定 $H.min$

```

1:  $A \leftarrow$  容量为 1、初值为  $null$  的数组;  $C \leftarrow 1$ ;  $D \leftarrow -1$  ▷  $C$  为当前容量
2: while  $H.roots \neq null$  do
3:   从  $H.roots$  摘下一个根  $x$ ;  $d \leftarrow x.degree$ 
4:   while  $d \geq C$  do
5:     将  $A$  扩容至  $2C$ , 保留原有元素, 新增位置置为  $null$ ;  $C \leftarrow 2C$ 
6:   end while
7:   while  $A[d] \neq null$  do
8:      $y \leftarrow A[d]$ ;  $A[d] \leftarrow null$ 
9:     if  $x.key > y.key$  then
10:      交换变量  $x, y$ 
11:    end if
12:     $y.parent \leftarrow x$ ;  $y.mark \leftarrow false$ ; 将  $y$  加入  $x$  的孩子链表
13:     $x.degree \leftarrow x.degree + 1$ ;  $d \leftarrow x.degree$ 
14:    while  $d \geq C$  do
15:      将  $A$  扩容至  $2C$ , 保留原有元素, 新增位置置为  $null$ ;  $C \leftarrow 2C$ 
16:    end while
17:  end while
18:   $A[d] \leftarrow x$ ;  $D \leftarrow \max(D, d)$ 
19: end while
20:  $H.min \leftarrow null$ 
21: for  $d = 0, 1, \dots, D$  do ▷  $D = -1$  时不执行循环
22:   if  $A[d] \neq null$  then
23:     将  $A[d]$  加入  $H.roots$ ; 其  $parent \leftarrow null$ 、 $mark \leftarrow false$ 
24:     if  $H.min = null$  或  $A[d].key < H.min.key$  then
25:        $H.min \leftarrow A[d]$ 
26:     end if
27:   end if
28: end for

```

以图 5.5 为例, 删除根 3 后, 9 和 11 成为新根。若依次处理根 6、8、9、11, 先将度数均为 0 的 8、9 合并, 得到以 8 为根的度数为 1 的树; 再将它与以 11 为根的树合并, 8 的度数增至 2; 最后与度数为 2 的根 6 合并, 得到度数为 3 的根 6。图 5.6 给出了整理前后的森林。

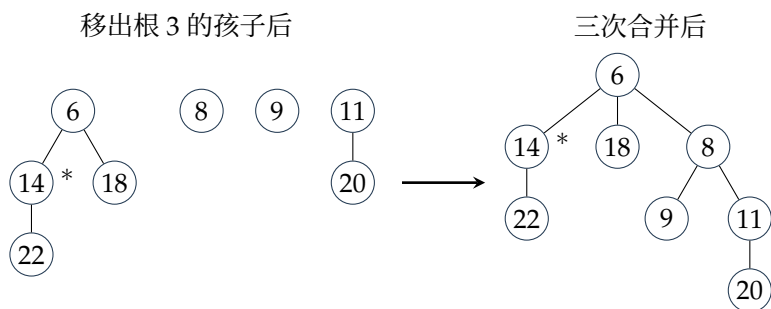


图 5.6 删除最小键 3 后, 依次合并根 8 与 9、8 与 11、6 与 8; 结点 14 的标记保持不变。

设删除前堆中共有 t 棵树，要删除的最小根为 z ，它有 d 个孩子。将这 d 个孩子逐个移入根链表需要 $O(d)$ 时间。原来的 t 个根中删去了 z ，又加入了它的 d 个孩子，所以接下来需要整理的根共有 $t - 1 + d$ 个。Consolidate 会逐个取出这些根，合并度数相同的两棵树。每次合并只需常数时间，而且使树的总数减少 1，因此合并次数不超过 $t - 1 + d$ ，处理这些树的总时间为 $O(t + d)$ 。辅助数组 A 的开销不难理解为 $O(D)$ ，因此一次 DeleteMin 的实际时间为 $O(t + d + D)$ 。这里的根数 t 可能很大。例如，从空堆连续插入 n 个元素后，会有 n 棵单结点树，紧接着的一次 DeleteMin 就需要处理其余 $n - 1$ 个根，花费 $\Theta(n)$ 时间。第 5.3.7 节会证明 DeleteMin 的摊还时间为对数级。

5.3.4 降低键值：切断与级联切断

把 x 的键降低为 k 后， x 与孩子之间仍然满足堆序，唯一可能违反堆序的是 x 与父结点之间的边。如果 x 已经是根，或者 k 仍不小于父结点的键，就无需改变树形；否则将 x 连同整棵子树从父亲处切下，加入根链表，这样就去掉了违反堆序的那条边。切下 x 后，还要处理父结点 y 失去孩子的情况。如果 y 是根，到此结束；如果 y 还没有标记，就给它加上标记，表示第一次失去孩子；如果 y 已经有标记，就把 y 也切下。此时 y 的父亲又失去了一个孩子，继续按同样的规则处理。这一过程称为级联切断，它保证基本结构中约定的失去孩子次数上限。

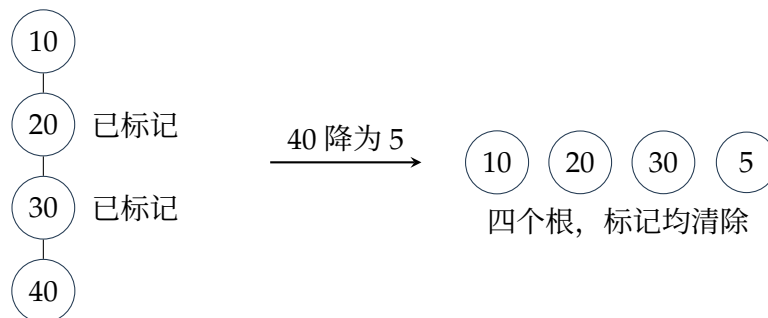


图 5.7 将 40 降为 5 后的级联切断过程；图中只画出相关结点，其余子树省略。

图 5.7 中，20 和 30 都带有标记。把 40 的键降为 5 后，先将它从 30 处切下；30 第二次失去孩子，需要从 20 处切下；接着 20 也第二次失去孩子，需要从 10 处切下。10 是根，过程结束。

先给出切断一个结点的辅助过程，它每次只摘下一个孩子，因此只需常数时间。

算法 5.8 Cut(H, x, y)

输入： $x.parent = y$ ，且 x 确实位于 y 的孩子链表中

输出： 将以 x 为根的子树移入根链表，清除 x 的父指针和标记

- 1: 从 y 的孩子链表摘下 x ； $y.degree \leftarrow y.degree - 1$
- 2: $x.parent \leftarrow null$ ； $x.mark \leftarrow false$ ；将 x 加入 $H.roots$

算法 5.9 FibonacciDecreaseKey(H, x, k)

输入： x 是 H 中的结点， $k \leq x.key$

输出： 将 x 的键改为 k ，恢复堆序及标记约束，并更新最小根

- 1: $x.key \leftarrow k$ ；同步更新记录中的键； $y \leftarrow x.parent$
- 2: **if** $y \neq null$ 且 $x.key < y.key$ **then**

```

3:  Cut( $H, x, y$ )
4:  while  $y.parent \neq null$  do
5:      if  $y.mark = false$  then
6:           $y.mark \leftarrow true$ ; break
7:      end if
8:       $p \leftarrow y.parent$ ; Cut( $H, y, p$ );  $y \leftarrow p$ 
9:  end while
10: end if
11: if  $x.key < H.min.key$  then
12:      $H.min \leftarrow x$ 
13: end if

```

若共切断 c 个结点, 则时间为 $O(c+1)$ 。单次降键可能引起很长的级联切断, 但除第一次切断外, 每次切断都会清除一个已有标记, 因此直观上摊还分析可以利用标记数的下降来抵消这部分成本。

5.3.5 删除指定结点

给定结点 x 后, 可以先将它的键降到小于所有普通键的特殊值 $-\infty$, 再执行 DeleteMin 即可, 其摊还成本就是这两次操作的摊还成本之和。

算法 5.10 FibonacciDelete(H, x)

输入: x 是 H 中的结点

输出: 删除 x 并返回其原记录, x 的指针失效

- 1: 保存 x 的原记录副本 r
- 2: FibonacciDecreaseKey($H, x, -\infty$)
- 3: FibonacciDeleteMin(H); **return** r

5.3.6 度数上界

DeleteMin 需要移出最小根的所有孩子, 并按度数整理各根, 因此必须控制一个结点最多能有多少孩子。

定理 5.2 (斐波那契堆的度数上界). 设斐波那契数列满足 $F_0 = 0, F_1 = 1$ 以及 $F_j = F_{j-1} + F_{j-2}$ ($j \geq 2$)。在从空堆按上述操作构造的斐波那契堆中, 度数为 k 的结点, 其子树至少有 F_{k+2} 个结点。因此, 含 $n \geq 1$ 个结点的堆中, 每个结点的度数至多为 $\lfloor \log_\varphi n \rfloor$, 其中 $\varphi = (1 + \sqrt{5})/2$ 。

证明. 设 x 目前有 k 个孩子, 按它们成为 x 的孩子的先后顺序记为 $y_1, \dots, y_k$ 。当 y_i 被接到 x 下方时, $y_1, \dots, y_{i-1}$ 都已经是 x 的孩子, 所以当时 x 的度数至少为 $i-1$ 。由于只有同度数的根才会合并, 当时 y_i 的度数也至少为 $i-1$ 。此后 y_i 至多失去一个孩子, 否则它就会被切走。因此, 现在各孩子的度数满足 $\deg(y_1) \geq 0, \deg(y_i) \geq i-2 \quad (i \geq 2)$ 。

这意味着, 度数大的结点必然包含足够大的子树。令 $size(x)$ 为以 x 为根的子树结点数 (包含 x)。我们对 $size(x)$ 作强归纳, 证明度数为 k 的结点满足 $size(x) \geq F_{k+2}$ 。 $k=0$ 时, 子树只有 x 自己, 恰

有 $1 = F_2$ 个结点。对于 $k \geq 1$ ，每棵孩子子树都比原树小，可以应用归纳假设： y_1 至少有一个结点，而 y_i 的度数至少为 $i - 2$ ，所以其子树至少有 F_i 个结点。加上根本身，得到

$$\text{size}(x) = 1 + \sum_{i=1}^k \text{size}(y_i) \geq 2 + \sum_{i=2}^k F_i = F_{k+2}.$$

最后一个等式来自 $\sum_{i=0}^k F_i = F_{k+2} - 1$ ，不难归纳证明。 $\square$

5.3.7 摊还复杂度分析

此前已经找出了两种可能成本很高的操作：DeleteMin 可能需要整理很多根，但合并之后根数会大幅减少；DecreaseKey 可能连续切断很多结点，但除第一次外，被切下的结点都带有标记，切断时这些标记会被清除。因此，我们自然希望用根数支付整理的费用，用标记数支付级联切断的费用。

令 $t(H)$ 为根数， $m(H)$ 为带标记结点的个数，取 $\Phi(H) = t(H) + 2m(H)$ 。切下一个带标记的结点时，根数增加 1，标记数减少 1，所以势能净减少 1，这就是标记前系数取 2 的原因。注意接下来的讨论中我们会忽略一些常数项，但不影响最终的界。

插入、查找与合并

Insert 的实际成本为 $O(1)$ ，同时新增一个无标记的根，故 $\Delta\Phi = 1$ ，摊还成本为 $O(1) + 1 = O(1)$ 。FindMin 不改变结构，势能变化为 0，实际成本和摊还成本都是 $O(1)$ 。Merge 将两个堆的根数、标记数分别相加，原来两个堆置空，因此所有堆的总势能不变，摊还成本也为 $O(1)$ 。

降低键值

先考虑发生了切断的情况，设一共切下 $c \geq 1$ 个结点。根数增加 c 。第一次切下的 x 可能没有标记，但其后的 $c - 1$ 个结点在切断前一定带有标记，切断后标记都被清除。过程结束时，至多给一个原先未标记的祖先加上标记，所以 $\Delta m \leq 1 - (c - 1) = 2 - c$ ， $\Delta\Phi \leq c + 2(2 - c) = 4 - c$ 。 c 次切断及其循环判断、收尾工作的实际成本至多为 $c + 1$ ，所以 $\hat{a} = a + \Delta\Phi \leq (c + 1) + (4 - c) = 5 = O(1)$ 。如果没有发生切断，只进行常数比较和赋值，根数、标记数均不变，同样只需 $O(1)$ 摊还时间。

删除最小值与删除指定结点

设 DeleteMin 开始前有 $n \geq 1$ 个结点、 t 个根，被删根有 d 个孩子。取 $D = \lfloor \log_\varphi n \rfloor$ ，由定理 5.2，被删根的度数和整理过程中各结点的度数都不超过 D 。根据操作部分的计数，实际成本满足 $a \leq t + d + D$ 。整理后各根的度数互不相同，只可能取 $0, 1, \dots, D$ ，所以新根数 $t' \leq D + 1$ 。这次操作只会清除标记，不会增加标记，故 $\Delta\Phi \leq t' - t \leq D + 1 - t$ 。因此，摊还成本满足 $\hat{a} = a + \Delta\Phi \leq (t + d + D) + (D + 1 - t) = d + 2D + 1 = O(\log n)$ 。

Delete 先执行一次 DecreaseKey，再执行一次 DeleteMin，两步的摊还时间分别为 $O(1)$ 和 $O(\log n)$ ，所以删除指定结点的摊还时间也是 $O(\log n)$ 。

5.3.8 几种堆的操作时间复杂度

下表汇总本讲涉及的几种堆。令 n 为堆中的元素数，合并时则取两个输入堆的元素总数，并记 $L = \log n$ ，带星号的是摊还时间上界，其余是最坏情况上界。

	二叉堆	左式堆	斜堆	二项堆	斐波那契堆
Insert	$O(L)$	$O(L)$	$O(L)^*$	$O(1)^*$	$O(1)$
Merge	$O(n)$	$O(L)$	$O(L)^*$	$O(L)$	$O(1)$
DeleteMin	$O(L)$	$O(L)$	$O(L)^*$	$O(L)$	$O(L)^*$
Delete	$O(L)$	$O(L)$	$O(L)^*$	$O(L)$	$O(L)^*$
DecreaseKey	$O(L)$	$O(L)$	$O(L)^*$	$O(L)$	$O(1)^*$

表 5.1 各类堆的操作时间复杂度，星号表示摊还时间。

5.4 习题

习题 5.1 (Combine 次数). 如果从空堆开始连续插入 n 个元素，总共会执行多少次 CombineTrees? 请给出精确表达式。

习题 5.2 (奇偶深度). 约定根的深度为 0，记二项堆中深度为偶数、奇数的结点数分别为 N_0, N_1 。证明 $N_0 - N_1$ 只能等于 0 或 1，并说明这个差值由元素总数的奇偶性如何决定。由此判断：当 $N_0 > N_1$ 时，元素总数是否可能为偶数？

习题 5.3 (用合并实现连续插入的问题). 解释为什么直接用通用合并算法实现从空堆开始的连续插入，不能保证常数摊还时间。

习题 5.4 (2-3-4 堆的设计). 设计一种支持合并操作的 2-3-4 堆。它采用类似 B+ 树的多路树结构，但不要求叶子中的键按大小排列。具体要求如下：

1. 每个叶子存放一个元素的键，各叶子之间没有键值顺序要求。
2. 每个内部结点 x 保存其子树中所有叶子键的最小值，根为内部结点时也遵守这一规定。
3. 所有叶子等深，每个内部结点有 2 至 4 个孩子，根还需保存树的高度。

整个堆放在内存中，无需考虑磁盘读写。请实现下列操作：对于含 n 个元素的堆，前五种操作都应在 $O(\log n)$ 时间内完成；Merge 也应达到这一时间界，其中 n 取两个输入堆的元素总数。

1. FindMin：返回存放最小键的叶子指针。
2. DecreaseKey：将给定叶子的键降低为指定值。
3. Insert：插入一个存放给定键的新叶子。
4. Delete：删除给定叶子。
5. DeleteMin：删除存放最小键的叶子。
6. Merge：合并两个堆，返回结果堆，并销毁原来的两个堆对象，合并后的结点继续保留。

实现方法可以自行选择，但需要说明如何保持结构性质，以及为什么满足上述时间要求。

习题 5.5 (堆加速 Prim 算法). 回忆求最小生成树的 Prim 算法，说明堆可以加速其中的哪一步、堆中的键应当存什么，并给出具体的更新方法。

5.5 参考答案

习题 5.1. 连续插入 n 个元素，共新建了 n 棵单结点树，而每执行一次 `CombineTrees`，树的总数就减少一。设 n 的二进制表示中有 $\text{popcount}(n)$ 个 1，最终就有这么多棵树，所以合并次数恰好为 $n - \text{popcount}(n)$ 。□

习题 5.2. 先考虑一棵 B_k 。由定理 5.1，深度为 d 的结点有 $\binom{k}{d}$ 个，所以当 $k \geq 1$ 时，偶数深度与奇数深度的结点数之差为 $\sum_{d=0}^k (-1)^d \binom{k}{d} = (1-1)^k = 0$ 。而 B_0 只有一个深度为 0 的根，差值为 1。因此，将堆中各棵树的差值相加，结果只取决于有没有 B_0 ：元素总数为偶数时，没有 B_0 ，故 $N_0 = N_1$ ；为奇数时，有一棵 B_0 ，故 $N_0 = N_1 + 1$ 。空堆两者都为 0，也满足结论。所以当 $N_0 > N_1$ 时，元素总数一定为奇数。□

习题 5.3. 因为通用合并算法每一次合并都要扫描所有的树，并不像连续插入时那样只扫描到第一次不需要进位就停止，因此连续 n 次插入的总时间为 $\Theta(n \log n)$ 。□

习题 5.4. 略。□

习题 5.5. 略。□

5.6 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第五讲。
2. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*, 第二版, 第 6.8 节 (二项队列)、第 11.2 节 (二项队列的摊还分析)。
3. Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein, *Introduction to Algorithms*, 第三版, 第 19 章 (斐波那契堆), Dijkstra 与 Prim 分别见第 24.3、23.2 节。
4. Daniel D. Sleator、Robert E. Tarjan, *Self-Adjusting Heaps*, *SIAM Journal on Computing*, 15(1), 1986, 第 4 节, 第 65 页 (斜堆的任意删除)。